

```

import math #
=====
# CONFIGURAÇÕES (Limites, Faixas de Validação e Fatores de Conversão) #
=====
# Baseado na variante do aluno: Cenário da Mecânica (Pastilha de Freio) CONFIG_SENSORES = {
"temperatura": { "min_raw": 0, "max_raw": 1023, "fator_conv": 600.0 / 1023.0, "limiar_alerta": 200.0,
"limiar_critico": 450.0, "unidade": "°C" }, "pressao": { "min_raw": 0, "max_raw": 1023, "fator_conv":
100.0 / 1023.0, "limiar_alerta": 15.0, "limiar_critico": 65.0, "unidade": "Bar" }, "vibracao": { "min_raw":
0, "max_raw": 1023, "fator_conv": 12.0 / 1023.0, "limiar_alerta": 3.0, "limiar_critico": 7.0, "unidade": "g"
} } #
=====
# MÓDULO 1: ENTRADA DE DADOS #
=====
def receber_medicoes_brutas(): """Retorna um conjunto simulado de medições brutas coletadas na
frenagem.""" return [ {"temperatura": 100, "pressao": 20, "vibracao": 100}, # Largada {"temperatura":
150, "pressao": 15, "vibracao": 300}, # Acelerando {"temperatura": -50, "pressao": 15, "vibracao":
450}, # ERRO: Fora da faixa {"temperatura": 220, "pressao": 10, "vibracao": 550}, # 200 km/h,
pastilha encostando {"temperatura": 250, "pressao": "Erro", "vibracao": 600}, # ERRO: Não numérico
{"temperatura": 600, "pressao": 700, "vibracao": 800}, # 850m: Alicates acionados {"temperatura": 800,
"pressao": 850, "vibracao": 900}, # Frenagem máxima {"temperatura": 950, "pressao": 800,
"vibracao": 950}, # Calor extremo {"temperatura": 900, "pressao": 750, "vibracao": 700}, # Quase
parando {"temperatura": 750, "pressao": 50, "vibracao": 50}, # 1000m: Parada total ] #
=====
# MÓDULO 2: VALIDAÇÃO E CONVERSÃO (Reaproveitamento de Funções) #
=====
def validar_medicao(valor_bruto, min_raw, max_raw): """Verifica se o valor é numérico e está dentro
da faixa fisicamente possível.""" if not isinstance(valor_bruto, (int, float)): return False if valor_bruto <
min_raw or valor_bruto > max_raw: return False return True
def converter_para_engenharia(valor_bruto, fator_conversao): """Converte o valor bruto lido do sensor
para a unidade de engenharia real.""" return valor_bruto * fator_conversao #
=====
# MÓDULO 3: PROCESSAMENTO MATEMÁTICO E ESTATÍSTICO #
=====
def calcular_estatisticas(valores_convertidos, total_bruto): """Calcula métricas estatísticas: min, max,
média, desvio padrão e contagens.""" validas = len(valores_convertidos) descartadas = total_bruto -
validas if validas == 0: return {"min": 0, "max": 0, "media": 0, "std_dev": 0, "validas": 0, "descartadas":
descartadas} v_min = min(valores_convertidos) v_max = max(valores_convertidos) media =
sum(valores_convertidos) / validas variancia = 0 if validas > 1: variancia = sum((x - media) ** 2 for x
in valores_convertidos) / (validas - 1) desvio_padrao = math.sqrt(variancia) return {"min": v_min,
"max": v_max, "media": media, "std_dev": desvio_padrao, "validas": validas, "descartadas":
descartadas}
def classificar_estados_e_eventos(valores, limiar_alerta, limiar_critico): """Classifica
medições e detecta a maior sequência consecutiva de estado crítico.""" estados = {"normal": 0,
"alerta": 0, "critico": 0} max_seq_critica = 0 seq_atual = 0 for v in valores: if v >= limiar_critico:
estados["critico"] += 1 seq_atual += 1 if seq_atual > max_seq_critica: max_seq_critica = seq_atual
else: seq_atual = 0 # Quebra a sequência consecutiva if v >= limiar_alerta: estados["alerta"] += 1
else: estados["normal"] += 1 return {"contagem_estados": estados, "maior_sequencia_critica":
max_seq_critica} #
=====
# MÓDULO 4: ORQUESTRAÇÃO DE CADA SENSOR #
=====
def analisar_sensor(leituras_brutas, chave_sensor, config): """Processa todas as etapas requeridas
para um único sensor.""" valores_convertidos = [] total_lido = len(leituras_brutas) # 1. Extração,
Validação e Conversão for leitura in leituras_brutas: valor_bruto = leitura.get(chave_sensor) if
validar_medicao(valor_bruto, config["min_raw"], config["max_raw"]): valor_convertido =
converter_para_engenharia(valor_bruto, config["fator_conv"])
valores_convertidos.append(valor_convertido) # 2. Cálculos Estatísticos estatisticas =
calcular_estatisticas(valores_convertidos, total_lido) # 3. Classificação e Detecção de Eventos
eventos = classificar_estados_e_eventos( valores_convertidos, config["limiar_alerta"],
config["limiar_critico"] ) # Consolidação do pacote de resultados return {"estatisticas": estatisticas,

```

```
"eventos": eventos} #
```

```
=====
# MÓDULO 5: SAÍDA DE DADOS (Relatório Diagnóstico) #
=====
```

```
def gerar_relatorio_diagnostico(resultados, configuracoes): """Formata e imprime os resultados finais
de maneira clara.""" print("=" * 60) print(" RELATÓRIO DE DIAGNÓSTICO: PASTILHA DE FREIO
(SRAD 1000) ") print("=" * 60) for sensor, config in configuracoes.items(): dados = resultados[sensor]
["estatisticas"] eventos = resultados[sensor]["eventos"] uni = config["unidade"] print(f"\n>> SENSOR
DE {sensor.upper()} ({uni})") print(f" Validar/Descartar: {dados['validas']} válidas |
{dados['descartadas']} descartadas (ruído/erro)") print(f" Estatísticas: Mín: {dados['min']:.2f} | Máx:
{dados['max']:.2f} | Média: {dados['media']:.2f} | Desvio: {dados['std_dev']:.2f}") est =
eventos["contagem_estados"] print(f" Classificação: Normal ({est['normal']}) | Alerta ({est['alerta']}) |
Crítico ({est['critico']})") print(f" Eventos: Maior sequência crítica detectada:
{eventos['maior_sequencia_critica']} medições consecutivas") print("-" * 60) #
```

```
=====
# FLUXO PRINCIPAL #
=====
```

```
def main(): """Função principal que organiza o fluxo de dados (Sem variáveis globais).""" # 1.
Receber medições brutas dados_brutos = receber_medicoes_brutas() # 2 e 3. Processar cada
grandeza usando a mesma lógica (Reuso/DRY) resultados = {} for sensor in
CONFIG_SENSORES.keys(): resultados[sensor] = analisar_sensor(dados_brutos, sensor,
CONFIG_SENSORES[sensor]) # 4. Emitir relatório gerar_relatorio_diagnostico(resultados,
CONFIG_SENSORES) # Execução do programa if __name__ == "__main__": main()
```